

PHASE 6 QUOTATIONS REPORT

DWELLERS

Find. Buy. Build.

Delivery record for Phase 6 of the Dwellers platform: quotations that complete the commercial conversation - an itemised, server-calculated GH¢ document, one central state machine, expiry and race protection, customer accept or decline - ending exactly at the payment boundary.

DELIVERY STATUS — COMPLETE

Verification 486 checks across six suites, all green
Browser E2E 28 + 6 steps, three viewports, zero overflow

Date September 2026

Prepared by Dwellers Engineering

DWELLERS · GHANA · GH¢

Table of Contents

1. Executive Summary	1
2. Objective and Scope	2
3. Existing Quote Foundation	3
4. Quote Architecture	4
5. Quote State Machine	5
6. Quote Creation and Provider Workflow	6
7. Quote Item Structure and Labour	7
8. Materials, Equipment, Transport and Other Costs	7
9. Calculation Algorithm and GHC Money Handling	8
10. Quote Validity	9
11. Customer Workflow	10
12. Notifications, Timeline and Audit Trail	11
13. API Endpoints and Database Changes	12
14. Security	13
15. Security and Acceptance Tests: IDOR, Tampering, Permissions, Expiry, Duplicates, Races.	
16. Browser E2E and Responsive Results	14
17. Performance and Known Issues	15
18. Deferred Features, Git Commits, Test Counts and the Phase 7 Recommendation	16

1. Executive Summary

109 Phase 6 verification checks, all passing	486 Total checks across the six suites	12 / 12 Critical acceptance tests passing	28 + 6 Browser E2E steps plus decline-flow steps
---	---	--	---

Phase 6 of the Dwellers platform completes the commercial heart of the product: a provider can now price the work a customer described in Phase 5, item by item, and the customer reviews a professional quotation document and decides. The full flow the owner's brief specified is real: JOB REQUEST, PROVIDER RESPONSE, CREATE QUOTE, ADD QUOTE ITEMS, CALCULATE TOTAL, SEND QUOTE, CUSTOMER NOTIFIED, CUSTOMER REVIEWS, ACCEPT OR DECLINE, PROVIDER NOTIFIED. The workflow then stops precisely where the brief draws the line: an accepted quotation is an acceptance, not a payment, and no payment, escrow, wallet or fee machinery was built. The work was performed against the 87-part Phase 6 requirements brief, and every part of that brief is accounted for in this report.

Three design decisions define the phase. First, the quotation totals are computed on the server, always: the browser may display running amounts for convenience, but the API schema has no field a client could use to declare an amount, and the service recalculates every line total, the subtotal, the discount and the total from the item data in integer pesewas - so the browser can never force a total of GH¢1.00 when the items sum to GH¢530.00. Second, one central state machine owns the quotation: clients submit actions, never statuses, and the existing Phase 2 status vocabulary (DRAFT, SUBMITTED, VIEWED, ACCEPTED, DECLINED, EXPIRED, WITHDRAWN) was preserved exactly as the brief required. Third, every decision is atomic: quote state, the job-request state move, the timeline event, the audit record and the notification land inside one transaction, guarded by conditional updates so a double click or two simultaneous decisions have exactly one winner.

The proof is behavioural. The new verify-phase6 suite adds 109 checks on top of the Phase 1 (48), Phase 2 (106), Phase 3 (73), Phase 4 (56) and Phase 5 (94) suites for a combined battery of 486 checks, all green, including the twelve critical acceptance tests of the brief: the exact Nsawam arithmetic (1×350 plus 4×25 plus $1 \times 80 = \text{GH¢}530.00$); send, view, accept and decline with their notifications and timeline events; the refusal to edit a sent quotation; the refusal to let a customer PATCH the total; expiry enforced at acceptance time; the IDOR matrix; the double-click send; and the concurrent-decision race. The full regression battery and a clean-database rebuild from all six migrations were executed, and a browser E2E walked the complete 28-step customer-to-provider-to-customer journey plus the decline flow at 390, 768 and 1440 pixel widths with zero console errors and zero horizontal overflow. The project now sits at the Phase 7 gate, awaiting the project owner's explicit approval.

2. Objective and Scope

Objective

Phase 6 exists to turn interest into agreement. After Phase 5, a customer could describe a job and a provider could say they were interested, but the commercial conversation had no artefact: no prices, no items, no validity, no decision. The objective of this phase was a complete, professional quotation workflow in which the provider prices the job without re-entering anything the request already knows, the system calculates every amount itself, the customer receives the quotation as a document they could show a real Ghanaian counterpart, and both sides see the decision in real time. The quotation must be professional enough to be printed and handed over, and honest enough that no element of it can be inflated or deflated from the client side.

Scope of this phase

In scope: quote eligibility rules, the quotation state machine, itemised line items across five kinds (labour, materials, equipment, transport, other costs), server-side calculation, fixed-amount discounts, quotation notes and terms, validity windows, expiry, send, view tracking, customer acceptance and decline with an optional reason, provider withdrawal, notifications, timeline events, the audit trail, customer and provider dashboards with filters, a per-request comparison list, print-friendly documents, responsive layouts and the full verification battery.

Out of scope by explicit instruction: payment gateways, mobile money, card payments, bank transfers, escrow, wallets, checkout, payment confirmation, refunds, commissions and platform fees (PART 51); WhatsApp, SMS and email channels (PART 52); tax types such as VAT, NHIL or GETFund levy (PART 16); quotation revision and versioning systems (PART 28); and AI comparison engines (PART 50). Each of these remains a clean seam for a later phase, documented in the deferred-features chapter.

Phase ledger

Phase	Scope	Status
1	Foundation, architecture, standards	Complete
2	Database and backend foundation	Complete
3	Authentication and role dashboards	Complete
4	Find / nationwide discovery	Complete
5	Request service / job requests	Complete
6	Quotations (this report)	Complete
7	Awaiting owner instruction	Gate

Table 2.1 - Phase ledger after Phase 6.

3. Existing Quote Foundation

The brief's first hard rule was to inspect the existing code before changing anything, and the inspection produced the single most important finding of the phase: the Phase 2 database build had already anticipated quotations completely. The Quote model existed with a unique human quoteNumber in the QT-prefixed format, the jobRequestId, providerId and a denormalised customerId for fast per-customer queries, component amount columns for labour, materials, equipment and other charges, discount, tax and total amounts, a currency column defaulting to GHS, a validUntil date, an estimated duration, terms, and the full status vocabulary the phase needed. QuoteItem existed with the exact five item kinds the brief lists, a name, description, quantityMilli for precise quantities, a unitLabel snapshot, unitPriceAmount in pesewas and a lineTotalAmount.

Equally important, the money policy was already in force. The schema carried an explicit comment, enforced since Phase 5, that quotation totals are always computed on the server from line items and that totals submitted by a frontend are never trusted. The finance helpers in src/lib/finance.ts provided formatCedi for display, toPesewas for converting decimal input with round-half-up, and lineTotalAmount for the sanctioned line-total arithmetic over integer thousandths. The permission catalogue from Phase 1 already contained commerce:quotes:request, commerce:quotes:submit and commerce:quotes:respond, granted to exactly the roles that should have them.

The consequence shaped the whole phase: only one genuine schema gap existed. The Quote model had terms but no dedicated notes column, and the brief explicitly requires quotation notes with a recommended maximum of five thousand characters. A single migration, 20260914_phase6_quote_notes, added the notes column as plain text. No other model, relation or index needed to change; no second quotation system, money system or notification system was created.

Foundation element	Source	Phase 6 action
Quote and QuoteItem models with QT reference	Phase 2 schema	Reused as-is
Integer-pesewa money policy and finance helpers	Phase 2 library	Reused as-is
commerce:quotes:* permission entries	Phase 1 matrix	Used, never altered
Job-request state machine and access resolver	Phase 5 engine	Reused; one transition added
Notification, audit, analytics, rate-limit pipelines	Phases 1-5	Extended with new vocabularies
Quotation notes column	-	New migration (the one gap)

Table 3.1 - What already existed and what Phase 6 added.

4. Quote Architecture

Phase 6 added one new domain module, `src/modules/quotes`, split into the same shape the platform uses everywhere. `quote-state.ts` is the pure, I/O-free state machine: statuses, actions, the transition table, the terminal set, the timeline event vocabulary and the validity comparison. `quote-service.ts` is the lifecycle engine: the eligibility ladder, the money calculation, the access resolution, and the create, edit, send, view, accept, decline, withdraw and expire operations, each ending in a transaction plus audit and analytics. Two thin helpers complete the module: a server-side mapper that projects service results into the view model every page shares, and request-scoped listing helpers used by the comparison panel.

The module deliberately reuses the request-side machinery rather than imitating it. Quote eligibility is defined against the same access resolver that Phase 5 uses for responding - so the notions of the targeted provider side, of business members who may act, and of opaque 404s for strangers are literally one implementation, not two that can drift. The API layer adds no logic of its own: all eight quotation endpoints delegate to the service through the Phase 1 handler factory, which applies rate limiting, authentication, the live account-status check, RBAC and Zod validation before the business code runs, and standardises the error envelope afterwards.

On the frontend, four shared components carry the whole feature. The quote form handles compose and preview with dynamic items; the quote document renders the professional view both sides see, with the decision dialogs; the quotes board renders the filtered lists; and the request-quotes panel renders the comparison view on a request detail. Role dashboards mount these through thin server pages - twelve provider routes across artisan, contractor and construction-company dashboards, plus the customer's list and detail. The CREATE QUOTE entry is wired into all five provider dashboards' request pages; supplier and equipment-provider roles, which the Phase 1 matrix deliberately excludes from quoting, see no button and no promise.

5. Quote State Machine

The machine preserves the Phase 2 vocabulary exactly as the brief demanded: the spec's SENT state is the existing SUBMITTED, and the spec's CANCELLED is the existing WITHDRAWN. The mapping is documented in the machine itself so no future reader invents a duplicate enum. Clients submit one of six actions - edit_draft, send, view, accept, decline, withdraw - and the server determines the resulting state; no API surface accepts a status value.

Action	Actor	From states	To state
edit_draft	Provider	DRAFT	DRAFT
send	Provider	DRAFT	SUBMITTED
view	Customer	SUBMITTED	VIEWED
accept	Customer	SUBMITTED, VIEWED	ACCEPTED
decline	Customer	SUBMITTED, VIEWED	DECLINED
withdraw	Provider	DRAFT, SUBMITTED, VIEWED	WITHDRAWN
(expiry)	System, lazily	SUBMITTED, VIEWED	EXPIRED

Table 5.1 - Actions, actors and permitted origin states.

Four properties make the machine trustworthy. It is total: every action is checked against both the actor kind and the origin state, and anything not listed is refused with a clear message such as this quotation has already been sent. It is honest: terminal states - accepted, declined, expired, withdrawn - accept nothing further, and the provider cannot accept or decline its own quote in any state. It is idempotent where it must be: the customer's first open claims the SUBMITTED-to-VIEWED move with a conditional update, so concurrent opens cannot duplicate the event or the provider notification. And it is lazy about expiry: there is no background job; a quotation past its validity is transitioned to EXPIRED by whichever read or decision path touches it first, with a single conditional claim so concurrent readers produce one event.

The machine was also extended where the brief required a decision rather than a feature. The job-request state machine gained one new transition, accept_quote, permitted only for the customer and only from the RESPONDED state, moving the request into its existing ACCEPTED status. This is the formal link between the two lifecycles: accepting a quotation commits the engagement, and the transition is invoked only by the quote service inside the acceptance transaction. Adding it changed nothing about Phase 5 behaviour - the Phase 5 suite was re-run green, with one assertion updated from six to seven job-request actions.

6. Quote Creation and Provider Workflow

Eligibility ladder (PART 2)

A provider cannot quote an arbitrary request. The creation operation runs a fixed ladder, every rung enforced server-side. The caller must be authenticated and hold the commerce:quotes:submit permission. The request must exist and the caller must be its targeted provider side - resolved through the same access resolver Phase 5 uses, so business members are handled by role. The request must be in RESPONDED with responseKind INTERESTED: a request the provider declined, a request still waiting, a cancelled request or an already-committed request is refused with a clear message. The provider account must still be live in the database - the suspended guard is applied again here, beneath the HTTP layer. Finally, the provider must not already have a quotation on this request: one quote per provider per request, because a withdrawn or declined quote is history, not a reusable template, and the brief forbids silent revisions.

Nothing re-entered (PART 3)

The CREATE QUOTE button appears on the provider's request detail only when the eligibility ladder would pass, and the form receives the customer, the service, the job title and the job location from the request - the provider never types them. On the server the linkage fields are again resolved from authorised records: the API schema has no customerId, providerId or jobRequestId field, and the stored quote's ownership chain is derived from the request row. The detail response exposes that resolved chain explicitly, which is how the verification suite proves that a tampered body cannot change who a quote belongs to.

Compose, preview, send (PART 26)

The provider flow is compose, preview, send. Compose offers dynamic item rows - add, edit, remove, with a configurable cap of fifty - plus notes, terms, an estimated duration, an optional fixed discount and a required validity date. The preview step renders the itemised quotation with the exact totals the server will compute, and the provider can go back to edit or send. Saving as a draft is possible at any point before sending; a draft is invisible to the customer. Sending claims the transition conditionally: a double click produces one send, one notification and one event, and the second attempt is told the quotation has already been sent.

7. Quote Item Structure and Labour

The line item (PART 8)

Every quotation line is a QuoteItem row with the five fields the brief requires: an item type, a description, a quantity, a unit and a unit price. The item types are exactly the vocabulary the Phase 2 schema already defined - LABOUR, MATERIAL, EQUIPMENT, TRANSPORT, OTHER - so no new types were invented. Quantities are stored as integer thousandths (2.5 bags becomes 2500) because the SQLite connector has no decimal type; prices are integer pesewas. Each item also carries a sort order so the document reads in the order the provider composed it, and an optional free-text unit label such as job, metres, day, trip or bag.

Labour (PART 5)

Labour items price the work itself. The brief's canonical example - labour, bathroom pipe repair, quantity 1, unit job, unit price GH¢350.00, amount GH¢350.00 - is implemented literally and asserted by the verification suite. The client may display the amount as the provider types, but the stored line total is computed by the server from the quantity in thousandths and the price in pesewas. In the browser E2E the provider entered exactly these three kinds of items and the document showed exactly the amounts the arithmetic demands.

Item	Kind	Qty (milli)	Unit price	Line total
Bathroom pipe repair	LABOUR	1000	35000	35000
PVC Pipe	MATERIAL	4000	2500	10000
Transportation	TRANSPORT	1000	8000	8000
			Subtotal	53000

Table 7.1 - The acceptance-test quotation as stored (integer pesewas).

8. Materials, Equipment, Transport and Other Costs

Materials (PART 6)

Material items cover consumables and parts, and the brief's worked examples are part of the verification suite: four metres of PVC pipe at GH¢25.00 yields GH¢100.00, and three PVC elbows at GH¢8.00 yield GH¢24.00. Fractional quantities are first-class because of the thousandths representation: the suite also

drives two and a half bags of cement at GH¢20.00 through the API and asserts exactly GH¢50.00 comes back, which is the case the naive float arithmetic would get wrong.

Equipment (PART 7)

Equipment items price tools and machinery where the job needs them - the brief's example is a pipe cutter for one day at GH¢50.00. Equipment is supported exactly as the model supports it: the kind exists, the dashboard type selector offers it, and its amounts roll into the equipmentAmount bucket on the Quote. The draft-edit verification adds precisely this item to a draft and asserts the recalculated totals.

Transport (PART 8) and other costs (PART 9)

Transport items price moving people and materials - one trip at GH¢80.00 in the acceptance test. Other costs cover legitimate extras such as a site inspection. The Quote model's denormalised buckets have no separate transport column, so transport and other roll up into otherChargesAmount while the line items remain the source of truth; the mapping is documented in the calculation code. The customer's document groups items under readable headings - Labour, Materials, Equipment, Transport, Other costs - so a five-line quote still reads like a business document rather than a database dump.

9. Calculation Algorithm and GH¢ Money Handling

The algorithm (PART 13/14/17)

One function, computeQuoteTotals, is the single place where money is derived. It takes the raw item inputs and an optional discount and produces every stored figure. Quantity is converted to integer thousandths and the price to integer pesewas before any multiplication; the line total uses the sanctioned round-half-up helper; the four component buckets are summed by kind; the subtotal is the sum of the buckets; the discount is converted and validated; and the total is subtotal minus discount. There is no floating-point money anywhere in storage or arithmetic - the only decimals are the boundary conversions at round half up, which is the documented Phase 2 policy.

The browser cannot corrupt this. The input schema has no amount, total, subtotal or status field at all, so a malicious payload carrying amount = 1 or total = 1 is stripped as an unknown key before the service runs; the suite posts such payloads and asserts the stored total equals the server calculation. The same applies to per-item fake amounts. The verification suite additionally replicates the client-side display logic and compares it with the stored values to catch display-only drift.

The money grid (PART 45)

Case	Expected	Result
GHC0.01 unit price	1 pesewa stored	Pass
GHC1.00, GHC100.00	100 / 10000 pesewas	Pass
GHC999,999.99 (ceiling)	99999999 pesewas	Pass
Multiple items, mixed kinds	Exact bucket sums	Pass
Discount GHC20 on GHC604	Total GHC584.00	Pass
Decimal quantity 2.5 x GHC20	GHC50.00 exactly	Pass
Negative price / negative discount	Rejected	Pass
Zero quantity	Rejected	Pass
NaN, Infinity over JSON	Rejected (422/400)	Pass
Unreasonably large quantity	Rejected	Pass

Table 9.1 - Money grid results (all server-verified).

Discount policy (PART 15) and tax (PART 16)

Discounts are fixed amounts, never percentages, and a discount that reaches or exceeds the subtotal is rejected at creation, at edit and over the API. Tax is deliberately absent: the taxAmount column exists and stays zero, and no VAT, NHIL or GETFund levy was invented. If Ghanaian tax configuration is required later it must arrive as a separate, controlled feature with its own migration and documentation, exactly as the brief prescribes.

10. Quote Validity

Every quotation carries a required validUntil date, and nothing in the platform is valid forever. The server validates the date at creation, at draft edit and again at send: a past date is rejected, a date more than seven hundred and thirty days ahead is rejected, and a draft that sat unused until its own validity passed cannot be sent - the provider must pick a future date first. The customer-facing document prints the validity prominently in the header block, and the acceptance dialog restates it before the customer commits.

Expiry is enforced where it matters. There is no background job to forget and no cron dependency to misconfigure; instead, any read of a sent or viewed quotation whose validity has elapsed lazily transitions it to EXPIRED inside a claimed transaction, writing the QUOTE_EXPIRED timeline event and the audit record. The acceptance path performs the same check before the machine transition: an expired quotation is expired first and then refused with the message the brief specifies - this quotation has

expired - and the customer sees the honest Expired state on the document. The suite proves both directions: time-travelled quotes are refused on accept and flipped on plain read.

11. Customer Workflow

Receiving and reviewing (PART 25/57)

The customer meets a quotation in three honest places: the QUOTE_SENT notification, the MY QUOTATIONS board and the quotations panel on the request itself. The document view is deliberately a business document: a DWELLERS header with the word Quotation, the reference in mono type, the sent date and the valid-until date, the provider, the customer, the job with its request reference, the service and location, the items grouped by kind, the subtotal, any discount, the total, the notes and terms, and the status badge - everything the brief's document-quality list requires, rendered from actual server data. A print button produces a clean print layout, and the print stylesheet removes the interactive chrome (PART 56), so the quotation can be handed over on paper today; a formal PDF generator remains future work.

Deciding (PART 29/30)

Accept and decline are never one-click accidents. Each opens an accessible confirmation dialog that restates the total, the provider, the job and the validity before the customer confirms; the decline dialog collects an optional reason of at most five hundred characters. Server-side, acceptance walks the brief's validation ladder: the customer owns the request and is the quote recipient, the quotation is in a decidable state, it has not expired, the request has not been cancelled or declined, and no incompatible quotation has already been accepted. Then one transaction updates the quote, moves the request through the accept_quote transition, writes the timeline event and creates the provider notification. Declining closes the quotation but deliberately leaves the request open in RESPONDED, so the customer can still accept a different provider's price - the comparison foundation of PART 49.

The customer cannot mutate what they are deciding on. The PATCH endpoint requires the submit permission that customers do not hold, so an attempted total change is refused with 403 before validation even runs; item edits, price changes, provider changes, expiry changes and ownership changes have no client-facing surface at all. The comparison list on the request detail shows every quotation side by side - provider, reference, total, validity, status - which is exactly the simple, honest foundation PART 50 asks for; no AI ranking, no price-only ordering.

12. Notifications, Timeline and Audit Trail

Notifications (PART 32/33)

Four notification types were added to the existing vocabulary - `QUOTE_SENT`, `QUOTE_VIEWED`, `QUOTE_ACCEPTED` and `QUOTE_DECLINED` - and every one is delivered through the Phase 5 notification architecture: the same Notification model, the same in-app channel, created inside the same transaction as the state change it announces. The customer is notified when a quotation arrives, in the shape the brief's example gives: the provider has sent you a quotation. The provider is notified when the customer opens the quotation (once), when it is accepted and when it is declined. No SMS or WhatsApp was built; the architecture remains channel-ready for them. Lazy expiry intentionally notifies nobody - it is a state transition the timeline shows, not a message.

Timeline (PART 34)

Quotation events live on the job-request timeline, not in a duplicate history. The event vocabulary added `QUOTE_CREATED`, `QUOTE_SENT`, `QUOTE_VIEWED`, `QUOTE_ACCEPTED`, `QUOTE_DECLINED`, `QUOTE_EXPIRED`, `QUOTE_WITHDRAWN` and `QUOTE_DRAFT_UPDATED`, each written in the same transaction as the change and each carrying the actor and role where a human performed it. The brief's auditable sequence - request submitted, provider responded, quote created, quote sent, quote viewed, quote accepted - is therefore one continuous trace a reader can walk on a single timeline, and the suite asserts exactly that sequence for the acceptance flow. Nothing is fabricated: events exist only where something happened.

Audit trail (PART 35)

The audit log records the full quotation action family: created, updated, item-added, item-updated, item-removed, sent, viewed, accepted, declined, expired and withdrawn, with actor, role, entity and minimal safe metadata - reference numbers, item counts, totals, and the decline reason where supplied. Item-level actions accompany a draft edit whenever the item set actually changed, with counts in the metadata. Passwords, secrets and personal data are never recorded; the notification bodies were swept by the suite for leaked emails. Analytics events for the quotation funnel - created, updated, sent, viewed, accepted, declined, expired, withdrawn - flow through the same privacy-safe discovery-event pipeline as every other funnel in the platform.

13. API Endpoints and Database Changes

Endpoints (PART 40)

Method and path	Permission	Purpose
POST /api/quotes	commerce:quotes:submit	Create a DRAFT (201)
GET /api/quotes	scoped by auth	Filtered, searched, paginated list with counts
GET /api/quotes/[id]	owner / provider / staff	Detail; customer open marks VIEWED
PATCH /api/quotes/[id]	commerce:quotes:submit	Edit DRAFT, totals recalculated
POST /api/quotes/[id]/send	commerce:quotes:submit	DRAFT to SUBMITTED, notify customer
POST /api/quotes/[id]/accept	commerce:quotes:respond	Customer acceptance ladder
POST /api/quotes/[id]/decline	commerce:quotes:respond	Customer decline, optional reason
POST /api/quotes/[id]/withdraw	commerce:quotes:submit	Provider withdrawal

Table 13.1 - The quotation API surface.

The withdraw route maps the brief's cancel action onto the project's existing WITHDRAWN vocabulary rather than inventing a synonym, exactly as PART 21 and 22 direct. No endpoint duplicates an existing one, and all eight run through the factory pipeline - rate limit, authentication, live account status, RBAC, Zod, then ownership, eligibility, the state machine, the transaction, audit, notification and the standard envelope, in that order.

Database changes (PART 23/61)

One migration: 20260914_phase6_quote_notes adds a nullable notes text column to the quotes table. That is the entire schema diff, and it exists only because the brief requires quotation notes while the Phase 2 model carried terms but not notes. No new tables, no new indexes, no changes to QuoteItem, JobRequest, Notification or AuditLog. The migration was applied with the non-destructive diff-and-deploy flow, and the clean-database rebuild rebuilt the whole database from all six migrations and reseeded it. Data integrity invariants hold: a quotation belongs to exactly one job request, one provider and one customer through foreign keys with cascade deletes; quote items exist only under a quote; monetary columns are constrained by the service to non-negative integers.

14. Security

The phase inherits the platform's security posture and adds the quotation-specific layers the brief lists. Authentication and the live account-status check run on every endpoint before any business code; RBAC is enforced against the Phase 1 matrix, so the submit and respond permissions are separate capabilities and a customer can never reach the edit or send endpoints at all. Ownership is structural: foreign callers receive the same opaque 404 as anonymous probes, whether they probe by id, by list or by action, so the existence of a quotation is never disclosed. The provider side sees a display-name-only customer, and no email, phone or account detail ever reaches a provider response.

Server-side integrity is the heart of the phase. Every stored amount is derived on the server from the item data; the client-facing schema has no amount, total, subtotal or status field, and unknown keys are stripped before validation. The eligibility ladder re-verifies the request's state and the provider's liveness beneath the HTTP layer, so a bypassed route changes nothing. Business members act only by role: a MEMBER can read the business's quotations but cannot create, edit, send or withdraw them. Suspended providers fail the live check and can neither create, send nor modify protected data. Every quotation action is audited, and notifications and audit rows never carry secrets.

Input hygiene follows the platform rules. Notes, terms, item names, descriptions and unit labels are length-capped, trimmed and stripped of control characters; the decline reason is capped at five hundred characters; the search query is a bounded plain-text token. All rendering is React text interpolation - no HTML injection surface exists in the document, the boards or the dialogs. Rate limiting uses the standard per-route buckets; the suite's flood checks confirm 429s arrive as designed.

15. Security and Acceptance Tests: IDOR, Tampering, Permissions, Expiry, Duplicates, Races

IDOR matrix (PART 43/75/76)

Ten cross-user scenarios were driven over live HTTP with real sessions, and all failed safely with 403 or 404 - never 200, never 500. Customer B cannot view, accept or decline Customer A's quotation, and Customer A's rows never appear in Customer B's list. Provider B cannot view, edit, send or withdraw Provider A's quotation, and cannot create a quotation on Provider A's request - no row is written. The same matrix is asserted at service level, where the refused cases raise the module's typed errors.

Price tampering (PART 44/73)

The tampering tests post exactly what a malicious browser would: fake amounts on each item, a fake subtotal, a fake total, a fake status and a foreign customerId in the create body; and, as the customer, a PATCH with total = 1. Every forged field is stripped as unknown to the schema, the stored total remains the server calculation (GH¢530.00 for the acceptance quote, GH¢23,000.00 for the mixed tamper payload), the status remains DRAFT, and the customer's PATCH is refused with 403 and the stored total untouched.

Permission, expiry, duplicate and race tests (PART 63/64/74/77/78)

Business permissions were exercised across OWNER, MANAGER and MEMBER: the owner and manager manage the business's quotation end to end while the member sees it read-only and is refused on send. A supplier role, which the matrix excludes from quoting, is refused at the service boundary even on its own targeted request. Suspended providers are refused over HTTP even with a valid session, because the live status check runs per request. The expiry test creates and sends a real quotation, moves its validity into the past, and proves acceptance is refused with the honest message and the row lands in EXPIRED. The duplicate-send test double-clicks send and counts exactly one notification and one timeline event. The race tests fire two concurrent accepts of one quote, and accept against decline, and prove exactly one transaction commits with exactly one decision event - the conditional-update guards hold under SQLite's serialised writes.

16. Browser E2E and Responsive Results

The 28-step journey (PART 79)

The full flow was walked in a real browser against the live server. The customer logged in, opened My Job Requests, opened the eligible request and saw the provider's response and the quotations panel. The provider logged in, opened the request, pressed Create quote, added labour, materials and transport, wrote notes, set the validity date, reviewed the preview totals and sent. The customer received the notification, opened the quotation through My Quotations, reviewed provider, job, all items, subtotal, total and validity, accepted through the confirmation dialog - which restates total, provider, job and validity - and the status became Accepted. The provider then received the acceptance notification, opened the quotation and saw the Accepted status with the exact total. Alongside the scripted journey, the PART 80 decline flow was walked end to end with an optional reason, ending with both sides seeing Declined and the provider notified.

Responsive checks (PART 53)

Viewport	Surfaces checked	Horizontal overflow	Critical console errors
390 px	Quote form, boards, document, request detail	None	None
768 px	Quote form, boards, document, request detail	None	None
1440 px	Quote form, boards, document, request detail	None	None

Table 16.1 - Viewport sweep results.

On mobile the form switches from the desktop table to stacked item cards and the boards switch to cards; on desktop the professional tables carry the density. The sweep measured scrollWidth against clientWidth on every page and found zero overflow at all three widths, and the console carried only development-mode notices - hot-reload messages and one next-auth URL warning that predates this phase - with zero critical errors. Evidence screenshots of the sent quotation, the customer document, the acceptance, the declined state and the mobile document are committed with the E2E work.

17. Performance and Known Issues

Performance (PART 60)

The data-access patterns follow the fixed-query discipline the platform uses everywhere. The quote detail loads the quote, its items, the request with service and location names, the provider display and the customer display in one include - no N+1 chains. The board list runs a fixed trio of queries: one page of rows with their joins, one count, and one grouped count for the status tabs. The request-scoped comparison list is a single indexed query on the jobRequestId foreign key, which the schema has indexed since Phase 2. Pagination is standard page and page-size with clamped bounds. Nothing in the phase introduced a background job, a queue or a cache to reason about.

Known issues

- Client display-mirror defect (found and fixed in E2E): the quote form's convenience totals mixed cedis and pesewas and displayed amounts one order of magnitude small (GH¢35.00 where GH¢350.00 was correct). The stored values were never wrong - the server always computes its own - and the fix aligns the mirror with finance.lineTotalAmount. The suite now asserts stored totals independently.
- Discount null handling (found and fixed in E2E): the form sends an explicit null to clear the discount, which the schema's optional() alone rejected with a 422. The schema now accepts null with clear-on-edit semantics.
- Environment repairs were required twice this phase (the recurring post-rebuild pattern): the .env lost AUTH_SECRET, and the development database had been created by db push without migration

history. Both were repaired - the secret regenerated, the database rebuilt from the six migrations and reseeded - and the rate-limit enablement flag was restored so the flood tests behave as in Phase 5.

- The dev-mode next-auth NEXTAUTH_URL warning remains visible in server logs; it predates this phase, is configuration-level, and has no functional effect on quotation behaviour.

18. Deferred Features, Git Commits, Test Counts and the Phase 7 Recommendation

Deferred features (PART 37)

- Payments in every form - gateway, mobile money, cards, bank transfer, escrow, wallet, checkout, confirmation, refunds, commission and platform fees - plus the Payment-model integration; the accepted quote is the clean handoff boundary.
- WhatsApp, SMS and email delivery for quotation notifications; the architecture is channel-ready.
- Quotation revisions and versioning: a sent quote is never silently edited, and no revision workflow was required for correctness this phase.
- Tax configuration (VAT, NHIL, GETFund levy) as a separate controlled feature.
- Print/PDF download beyond the print stylesheet; the document is print-friendly today and a formal generator remains future work.
- Advanced comparison: ranking or AI assistance over multiple quotations; the honest comparison table is the deliberate Phase 6 foundation.
- Equipment-rental workflows, project management, full messaging UI and analytics dashboards - unchanged from earlier phase deferrals.

Git commits (PART 38)

Commit	Content
8db5ba2	Quote engine: central state machine, lifecycle service, quotes API, vocabulary extensions, notes migration
70ff26e	Repo hygiene: untrack transient tool-results and upload artifacts
d5dd265	Quotation UI: form with preview, customer document, boards, CREATE QUOTE wiring
8a9c487	verify:phase6 suite (109 checks) + engine hardening from test findings
e25f7683	Form money-display fix + decline-null schema fix + E2E evidence screenshots
9332f20	Documentation: QUOTATIONS.md, architecture, database and README updates
(this)	Phase 6 report PDF and final ledger commit

Table 18.1 - Phase 6 commit ledger (in order).

Exact test counts (PART 39)

Suite	Checks	Scope
verify (Phase 1)	48	Foundation: env, fonts, security headers, health
verify:phase2	106	Domain, storage, messaging, authorization
verify:phase3	73	Authentication, onboarding, dashboards
verify:phase4	56	Discovery engine, acceptance, privacy, SEO
verify:phase5	94	Job-request lifecycle and security
verify:phase6	109	Quotation lifecycle and security
Total	486	All six suites green, dev and clean databases

Table 18.2 - Combined verification battery (fresh runs, all green).

The clean-database test rebuilt the database from all six migrations, reseeded it, and re-ran every suite green on the fresh data, confirming discovery, job requests, provider responses and quotations end to end. TypeScript strict compilation and ESLint report zero errors across the repository.

Phase 7 recommendation (PART 40)

The natural next phase is the one the product has been pointing at since the first quotation was accepted: payments. The clean boundary is already in place - an accepted quotation with its committed request, its audit trail and its timeline - so a payment phase can consume `Quote.status ACCEPTED` without touching the quotation engine: mobile money first, given the Ghanaian market, then cards and bank transfer, with escrow and platform fees as deliberate, separately reviewed decisions. If the owner prefers to broaden the marketplace before monetising, the next most valuable increments are quotation revisions for negotiation,

messaging between the two sides of an accepted engagement, or reviews once jobs complete. As always with this project: the gate belongs to the owner, and Phase 7 does not start without explicit approval.

Handover status

The working tree is clean and the phase is delivered in the same state as every previous one: source, tests, evidence screenshots, documentation and this report are committed, and the running system is the one that was verified. The database carries the six migrations in order, the seed reproduces the demo world from scratch, and the eight command-line suites re-prove the whole platform from a clean checkout in minutes. A reviewer who wants to confirm any single claim in this report can do so by running the named suite - every check prints its own pass and failure, and none of the numbers in these pages were transcribed by hand from memory.

For the project owner, the practical reading is simple. A Ghanaian customer can now find a provider anywhere in the country, describe the job with photos, receive an itemised quotation in cedis, compare it against other offers on the same request, and accept or decline it - and the provider's price can be trusted because no browser anywhere in the flow was ever allowed to state an amount. Everything beyond that point - collecting the money, holding it in escrow, settling the provider, taking the platform's fee - is deliberately unbuilt ground, marked by an accepted quotation and a committed request. That is the door Phase 7 opens, and it opens only on the owner's word.